

# torch.fx.experimental.symbolic\_shapes.rebind\_unbacked

[https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic\\_shapes.rebind\\_unbacked.html](https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_shapes.rebind_unbacked.html)

## Description:

Suppose we are retracing a pre-existing FX graph that previously had fake tensor propagation (and therefore unbacked SymInts). When we retrace, we re-propagate fake tensors, which results in new unbacked SymInts. When this happens, we need to tell the shape environment about the equivalence of the old and new unbacked SymInts. Pass us the old torch.fx.Node (which has the old binding information) and the new result (which we can extract the new unbacked SymInts out from).

## Function Signature

---

```
torch.fx.experimental.symbolic_shapes.rebind_unbacked(shape_env, n, result)[source]
```

# Detailed Documentation

---

## Documentation

Suppose we are retracing a pre-existing FX graph that previously had fake tensor propagation (and therefore unbacked SymInts). When we retrace, we re-propagate fake tensors, which results in new unbacked SymInts. When this happens, we need to tell the shape environment about the equivalence of the old and new unbacked SymInts. Pass us the old `torch.fx.Node` (which has the old binding information) and the new result (which we can extract the new unbacked SymInts out from).

Suppose we are retracing a pre-existing FX graph that previously had fake tensor propagation (and therefore unbacked SymInts). When we retrace, we re-propagate fake tensors, which results in new unbacked SymInts. When this happens, we need to tell the shape environment about the equivalence of the old and new unbacked SymInts. Pass us the old `torch.fx.Node` (which has the old binding information) and the new result (which we can extract the new unbacked SymInts out from).